

# Informe Profesional de Auditoria Tecnica

Lavanderia Exclusiva - Sistema de registro, recoleccion, produccion, inventario y nomina

Preparado por: Analisis senior de software

Fecha: 30 de julio de 2026

Resultado general: sistema operativo con buena base funcional, pero requiere endurecimiento de seguridad, auditoria de datos, paginacion y simplificacion de interfaz antes de crecer a gran escala.

| Area              | Estado                          | Comentario ejecutivo                                                                                                            |
|-------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Seguridad         | Media - requiere endurecimiento | Roles y CSRF estan bien encaminados; falta cerrar registro publico, cookies seguras y codigo empresarial sin valor por defecto. |
| Datos y auditoria | Media - alta prioridad          | Hay validaciones y transacciones, pero existen cascadas de borrado y renumeracion de ordenes que pueden afectar trazabilidad.   |
| Interfaz          | Operativa, pero cargada         | Funciona para usuarios entrenados; para personas de 60 a 75 anos conviene modo guiado, letra mayor y menos modales.             |
| Escalabilidad     | Limitada por consultas y vistas | Hay muchas consultas con get() y vistas de mas de 1000 lineas; se recomienda paginacion, filtros y modularizacion.              |
| Pruebas           | Buena base                      | 77 pruebas pasaron con 328 assertions; falta ampliar cobertura de concurrencia, auditoria y permisos negativos.                 |

# 1. Resumen ejecutivo

El sistema revisado es una aplicacion Laravel 12 con Blade, Tailwind, Alpine, sesiones en base de datos, cola en base de datos y configuracion preparada para PostgreSQL/Supabase. El alcance observado cubre usuarios, roles, catalogos, recolectores, clientes, facturas, produccion manual, incongruencias, pagos, gastos y reportes por quincena.

La base funcional es solida: las pruebas automatizadas pasan, el build de frontend compila y los flujos principales tienen validaciones. Sin embargo, para operar con crecimiento real y datos financieros sensibles, se recomienda una fase de estabilizacion antes de ampliar usuarios, volumen de ordenes o sedes.

**Verificacion ejecutada: php artisan test paso con 77 pruebas y 328 assertions. npm run build genero assets de produccion correctamente.**

## 2. Arquitectura actual

- Framework: Laravel 12, PHP 8.2+, Blade, Tailwind CSS y Alpine.js.
- Base de datos: configuracion orientada a PostgreSQL/Supabase, con fallback local usado por pruebas.
- Modulos principales: autenticacion, panel admin, recolector, clientes, catalogos, produccion, reportes, gastos, PQRS e incongruencias.
- Servicios existentes: numeracion de ordenes, auditoria de facturas, validacion de produccion, codigo empresarial, bloqueo de dispositivos y cache de dashboard.

Observacion senior: la aplicacion ya empezo a extraer logica a servicios, especialmente en validacion y numeracion. El siguiente paso natural es reducir la concentracion de responsabilidades del controlador administrativo y dividir vistas grandes en componentes reutilizables.

## 3. Seguridad

### Fortalezas

- Rutas administrativas protegidas por autenticacion, estado activo, rol admin/programador y throttle.
- CSRF presente en formularios Blade revisados.
- Bloqueo por dispositivo despues de intentos fallidos de codigo empresarial o login.
- Separacion de permisos para recolector, usuario, admin y programador.

### Riesgos y acciones

| Riesgo                            | Impacto                                             | Accion recomendada                                                                    |
|-----------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------|
| Registro publico activo           | Cuentas no autorizadas o confusion operativa.       | Cerrar registro despues del primer usuario o exigir invitacion/codigo administrativo. |
| Codigo empresarial por defecto    | Valor conocido si no se regenera.                   | Forzar generacion inicial y bloquear acceso si no existe codigo configurado.          |
| Cookies no endurecidas en ejemplo | Riesgo de sesion insegura si se copia a produccion. | SESSION_ENCRYPT=true,<br>SESSION_SECURE_COOKIE=true,<br>APP_DEBUG=false.              |
| Throttling fail-open              | Si cache falla, se permite la solicitud.            | Mantener fail-safe para disponibilidad, pero monitorear fallos de cache y alertar.    |

## 4. Base de datos y consistencia

El modelo de datos cubre las entidades clave del negocio: prendas de produccion, prendas de recolector, clientes, facturas, detalles, produccion, historial, gastos, pagos e incongruencias. Hay relaciones y transacciones en varios puntos criticos, lo cual es positivo.

### Puntos criticos

- Borrados en cascada sobre informacion financiera y operativa deben reemplazarse por conservacion historica. En lavanderia, nomina y facturacion, borrar datos reales rompe auditoria.
- Los numeros de orden deben ser inmutables. El sistema no deberia reenumerar ordenes existentes despues de una eliminacion; debe anular/cancelar y conservar el consecutivo.
- El cruce de prendas por nombre normalizado es util como solucion rapida, pero a escala debe existir una tabla de equivalencias entre catalogos.
- Agregar indices compuestos para consultas frecuentes: fecha + estado, recolector + fecha, quincena\_pago + estado, user + fecha, prenda + fecha.

## 5. Flujos del negocio

### Recolector

- Crea cliente, selecciona prendas activas, registra colores, cantidades, precios permitidos y genera orden.
- Valida colores por cantidad, evita prendas duplicadas y evita prendas inactivas.
- La comunicacion por WhatsApp esta contemplada y no rompe el guardado si falla.

### Lavandero

- Registra manualmente cantidades lavadas por prenda en cierre diario.
- La validacion compara prendas recibidas vs reportadas y genera faltantes/sobrantes.
- Los pagos se calculan sobre cantidades validadas o aprobadas.

### Admin y programador

- Gestionan usuarios, catalogos, facturas, gastos, reportes, incongruencias, cierres y dispositivos.
- Las acciones de habilitar/deshabilitar prendas quedan limitadas a admin/programador.

## 6. Hallazgos priorizados

| Prioridad | Hallazgo                                        | Impacto                                                                                                                                                           | Recomendacion                                                                                           |
|-----------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Alta      | Registro publico disponible                     | Las rutas /register siguen activas. Aunque los usuarios posteriores entran como usuario, en operacion real aumenta superficie de ataque y cuentas no autorizadas. | Cerrar registro publico tras primer usuario o exigir codigo de invitacion administrado.                 |
| Alta      | Borrados en cascada en datos operativos         | Algunas relaciones eliminan producciones, facturas o historial cuando se borra usuario, cliente o prenda. Esto no es ideal para nomina, inventario ni auditoria.  | Migrar a eliminacion logica, restrictOnDelete o nullOnDelete con campos historicos preservados.         |
| Alta      | Renumeracion de ordenes al borrar               | El servicio de numeracion reajusta facturas posteriores. A nivel contable, un numero de orden emitido debe ser inmutable.                                         | No renumerar. Cancelar/anular ordenes y conservar trazabilidad con motivo y usuario.                    |
| Alta      | Configuracion insegura si se copia .env.example | APP_DEBUG=true, SESSION_ENCRYPT=false y SESSION_SECURE_COOKIE=false aparecen en ejemplo. En produccion esto debe ser cerrado.                                     | Crear .env.production.example con valores seguros y checklist de despliegue.                            |
| Media     | AdminController concentra demasiada logica      | Dashboard, usuarios, facturas, reportes, incongruencias, produccion y finanzas viven en un controlador grande.                                                    | Extraer servicios/controladores por dominio: usuarios, facturas, reportes, incongruencias y dashboard.  |
| Media     | Consultas sin paginacion                        | Dashboard, recolector, produccion y PQRS cargan colecciones completas con get(). Con miles de registros, la respuesta bajara.                                     | Paginar, filtrar por periodo y agregar indices compuestos.                                              |
| Media     | Cruce de prendas por nombre                     | La validacion compara prendas recibidas/reportadas con normalizacion de texto. Cambios de nombre, tildes o duplicados pueden generar falsos faltantes/sobrantes.  | Crear tabla de equivalencias entre recolector_prendas y prendas de produccion.                          |
| Media     | Interfaz saturada para usuarios mayores         | Recolector y admin superan mil lineas de Blade y concentran muchas acciones. Esto aumenta carga cognitiva.                                                        | Dividir en pasos: Cliente, Prendas, Resumen, Guardar. Usar tipografia mayor y botones primarios claros. |

## 7. Interfaz para usuarios de 30 a 75 anos

El sistema tiene una interfaz visual moderna, con tarjetas, botones grandes y feedback por mensajes. Para usuarios de 30 a 50 anos con entrenamiento basico puede ser efectiva. Para usuarios de 60 a 75 anos se recomienda reducir densidad, aumentar tamano de letra y convertir flujos largos en pasos guiados.

| Area       | Situacion actual                                                         | Mejora propuesta                                                                                            |
|------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Recolector | Pantalla larga con cliente, prendas, colores, gastos, estatus y ordenes. | Separar en flujo de 4 pasos: Cliente, Prendas, Resumen, Guardar. Mantener historial en pantalla secundaria. |

| Area          | Situacion actual                                     | Mejora propuesta                                                                                                  |
|---------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Lavandero     | Tabla simple por prenda, buena para registro manual. | Aumentar input de cantidad, agregar botones + y -, agrupar prendas frecuentes y confirmar total antes de guardar. |
| Admin         | Panel extenso con muchos modales.                    | Crear menu lateral por modulos: Usuarios, Facturas, Produccion, Reportes, Incongruencias, Catalogos.              |
| Accesibilidad | Muchos textos text-xs/text-sm.                       | Base de 16px, alto minimo de botones 44px, contrastes fuertes, etiquetas persistentes.                            |

## 8. Escalabilidad y rendimiento

- Reemplazar get() masivos por paginate(), cursorPaginate() o filtros por periodo.
- Agregar busqueda server-side para clientes, facturas y usuarios.
- Separar calculos de dashboard en consultas agregadas o jobs programados.
- Usar Redis para cache, rate limiting y colas si aumenta el trafico.
- Medir tiempos con logs de consultas lentas y crear indices basados en uso real.

## 9. Hoja de ruta recomendada

| Periodo       | Objetivo                 | Entregables                                                                                                                        |
|---------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 0-7 dias      | Endurecimiento inmediato | Cerrar registro publico, exigir cookies seguras, regenerar codigo empresarial, limpiar cache y documentar variables de produccion. |
| 1-3 semanas   | Auditoria de datos       | Eliminar renumeracion de ordenes, reemplazar cascadas criticas, agregar soft deletes y tabla de eventos/auditoria.                 |
| 3-6 semanas   | Escalabilidad operativa  | Paginacion, filtros por quincena, indices compuestos, jobs para tareas pesadas y cache real con Redis si crece el trafico.         |
| 6-10 semanas  | UX para adopcion         | Modo operario senior, pasos guiados, confirmaciones claras, menos modales, pruebas con usuarios de 60-75 anos.                     |
| 10-14 semanas | Arquitectura modular     | Separar controladores, crear servicios de nomina/inventario/reportes y consolidar pruebas de permisos, pagos y concurrencia.       |

## 10. Conclusiones

El programa esta en una etapa saludable para operacion controlada: compila, las pruebas pasan y los flujos principales responden a necesidades reales del negocio. El mayor riesgo no es que el sistema no funcione hoy, sino que crezca sobre decisiones que luego sean dificiles de corregir: borrados en cascada, renumeracion historica, pantallas demasiado densas y consultas sin paginacion.

**Recomendacion senior: estabilizar seguridad, auditoria, paginacion y experiencia de usuario antes de escalar operaciones.**

## Anexo tecnico

### Referencias internas revisadas

| Tema                | Archivo                                      | Observacion                                                                   |
|---------------------|----------------------------------------------|-------------------------------------------------------------------------------|
| Rutas admin         | routes/web.php                               | Grupo protegido por auth, activo, rol:admin,programador y throttle.admin.     |
| Registro publico    | routes/web.php                               | Rutas /register activas.                                                      |
| Produccion validada | app/Models/Produccion.php                    | Campos cantidad_validada, total_validado, estado_validacion y scope pagables. |
| Validacion cruzada  | app/Services/ProduccionValidationService.php | Cruce de recibidas vs reportadas por fecha y prenda.                          |
| Numeracion ordenes  | app/Services/NumeroOrdenService.php          | Transacciones con lockForUpdate, pero reajuste historico riesgoso.            |
| Recolector UI       | resources/views/recolector/index.blade.php   | Pantalla interactiva grande con Alpine y modo tactil.                         |
| Admin UI            | resources/views/admin/dashboard.blade.php    | Panel extenso con multiples modales y acciones.                               |
| Configuracion       | .env.example                                 | Valores de desarrollo que no deben llegar a produccion.                       |

### Comandos de verificacion ejecutados

- php artisan test - resultado: 77 pruebas pasadas, 328 assertions.
- npm run build - resultado: compilacion Vite de produccion correcta.

Nota: Este informe evalua el estado del codigo y configuracion local revisada en el workspace. La configuracion final en Supabase/produccion debe validarse contra variables reales de despliegue, politicas de backups y permisos de base de datos.